



CEBU INSTITUTE OF TECHNOLOGY
UNIVERSITY

IT342 - G5

System Integration and Architecture

Software Test Plan Creation

Project Title: BayanLink: Barangay Services E-Portal

Prepared By: Jian Marc Ceniza

Date: May 08, 2026

Status: Final

Project: BayanLink: Vertical Slice Architecture Refactor

This document outlines the comprehensive Software Test Plan validating the core business logic, functional requirements, and automated execution metrics of the BayanLink system. It covers both the Spring Boot backend layer and the React frontend interface.

1. Functional Requirements Coverage

Req ID	Functional Requirement Module	Description	Coverage Status
FR-01	Authentication (Auth)	System must allow residents and officials to securely register and log in, with strict validation for @gmail.com domains and strong passwords. Guest access must be supported.	100% Covered
FR-02	Profile Management (User)	System must map a generic User entity to specific Resident demographics. Automatic age calculation from birthdate and edit-restrictions for Guests must be enforced.	100% Covered
FR-03	Document Requests (Document)	Residents must be able to submit document requests with purpose, urgency, and ID uploads. Officials must be able to transition document status (e.g., to READY_FOR_PICKUP).	100% Covered
FR-04	Community Scheduling (Appointment)	Residents must be able to book service appointments by selecting dates and predefined time slots. Approved appointments must trigger an automated email notification.	100% Covered
FR-05	Announcements (Announcement)	Officials must be able to author and broadcast announcements. The system must display these announcements in descending order of creation.	100% Covered

2. Test Cases

Test Case ID	Target Feature	Test Scenario	Expected Result
TC-AUTH-01	Registration	Submit registration with weak password (missing symbols/numbers).	System rejects submission and displays validation error message.
TC-AUTH-02	Login	User authenticates with valid @gmail.com email and correct password.	System grants access, displays success toast, and routes to Dashboard.
TC-PROF-01	Profile Edit	User updates profile details (e.g., address, occupation).	Data is saved; child Resident entity is updated/created; UI context updates.
TC-DOC-01	Document Request	Resident submits request with missing mandatory fields (e.g., missing ID).	Form submission blocked; validation toast displays required fields error.
TC-APP-01	Scheduling	Resident selects service, date, and morning time slot, then submits.	Appointment saved; reference number generated; summary updates dynamically.
TC-APP-02	Approval Workflow	Official approves pending appointment request.	Database status updates to APPROVED; approval email dispatched to Resident.

3. Test Scripts / Test Steps

The following procedures outline the steps required to manually verify system behavior for all functional modules prior to automated execution.

TS-01: Registration Validation (Weak Password)

1. Navigate to the "Register" route (/register).
2. Enter a valid email address (e.g., resident@gmail.com), First Name, and Last Name.
3. In the Password and Confirm Password fields, enter a weak string lacking numbers or special characters (e.g., weakpassword).

4. Click the "Create Account" button.
5. Expected Result: The system intercepts the submission and displays a toast error: "Password needs at least 1 number and 1 special character." The user is not registered.

TS-02: Successful Resident Login

1. Navigate to the "Login" route (/login).
2. Enter a valid, registered @gmail.com email address and the correct password.
3. Click the "Sign In" button.
4. Expected Result: A success toast displays "Login success!". After a 1000ms delay, the system successfully routes the user to the /dashboard.

TS-03: Profile Edit and Dynamic Calculation

1. Log in as a Resident and navigate to the "Profile" route (/profile).
2. Click the "Edit Profile" button to unlock the form fields.
3. Update the "Birthdate" field to a date exactly 20 years in the past.
4. Verify that the "Age" read-only field automatically calculates and displays "20".
5. Click "Save Changes".
6. Expected Result: A success toast displays "Profile updated successfully!", the database successfully updates (or cascades a new Resident record), and the user's display name updates dynamically.

TS-04: Document Request Validation (Missing ID)

1. Navigate to the "Document Requests" route (/documents).
2. Fill out Full Name, Select Document Type (e.g., Barangay Clearance), and Purpose.
3. Intentionally leave the "Valid ID Requirement" and "Upload ID Image" fields empty.
4. Click the "Submit" button.
5. Expected Result: The system blocks the API call and displays the error: "Please fill in all required fields and upload an ID."

TS-05: Resident Appointment Booking

1. Navigate to the "Community Scheduling" route (/appointments).

2. Select "Clinic Check-up" from the Service dropdown.
3. Click on a future date in the Calendar input.
4. Select the "10:00 AM" button from the Morning Time Slots array.
5. Verify that the readonly Summary text area dynamically displays the selected service, date, and time.
6. Click the "Submit" button.
7. Expected Result: A success toast displaying "Appointment booked! Ref: [REF-NUM]" appears, the form resets, and the payload is sent to the backend.

TS-06: Official Approval Workflow & Email Trigger

1. Log in using an Official account and navigate to the pending appointments queue.
2. Locate the pending appointment request generated in TS-05.
3. Click the "Approve" action button for that specific record.
4. Expected Result: The database status successfully transitions to APPROVED, and the system triggers the EmailService to dispatch an automated notification to the Resident's registered email address.

4. Automated Test Cases

To ensure continuous integration and prevent regression, the following automated test suites have been implemented and executed, achieving a **100% Pass Rate across 29 total tests**.

A. Backend Automated Tests (JUnit 5 & Mockito)

Scope: Service layer business logic, data persistence mapping, and mock repository integration. (*Total: 10 Tests*)

- **ProfileServiceTest (3 tests):** Verifies Resident demographics cascading, profile fetching, and "User Not Found" exception handling.
- **DocumentRequestServiceTest (2 tests):** Confirms strict typing conversions (Long vs Integer) and document status transitions (e.g., READY_FOR_PICKUP).
- **AnnouncementServiceTest (2 tests):** Validates Author User mapping and ordered retrieval queries.

- **AppointmentServiceTest (3 tests):** Checks Official approval logic, NPE prevention on missing mocks, daily schedule fetching, and EmailService triggers.

B. Frontend Automated Tests (Jest & React Testing Library)

Scope: Component rendering, simulated user events (fireEvent), context mocking, React Router navigation, and intercepted Fetch API calls. *(Total: 19 Tests)*

- **Register.test.js (3 tests):** Confirms password strength validation, password matching parity, and successful payload generation on form submit.
- **Login.test.js (4 tests):** Validates layout rendering, domain restrictions (@gmail.com), guest context bypassing, and simulated routing delays.
- **ResidentAppointments.test.js (5 tests):** Asserts dynamic state updates within the summary box, required field interception, form clearing, and JSON payload mapping to the mocked API.
- **Profile.test.js (4 tests):** Verifies context initialization, guest-user UI disabling, and automatic mathematical age calculation based on user birthdate inputs.
- **ResidentDocuments.test.js (3 tests):** Checks useEffect lifecycle hooks for data fetching on component mount, removal logic, and form constraints.